

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules 115 Bug 15 Code Smell 100

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 🔍

Extension identifiers should comply with a naming convention	Code Smell
Local identifiers should not start with underscore	Code Smell
If-null operator should be preferred	Code Smell
Null-aware operators should be preferred	Code Smell
Generic function type aliases should be preferred	Code Smell
Type parameters should not shadow other type parameters	Code Smell
"static final" declarations should be "const" instead	Code Smell
Interpolation should be used instead of String concatenation	Code Smell
Collection literals should be preferred	Code Smell
Conditional assignment should be preferred	Code Smell
"this" should only be used when required	Code Smell
Exceptions should not be ignored	

Extension identifiers should comply with a naming convention

Analyze your code

Consistency - Identifiable Maintainability ▾

Code Smell Minor ⓘ convention

Extension identifier, when present, should be in PascalCase.

Why is this an issue? How can I fix it? More Info

Code examples

Noncompliant code example

```
extension my_extension on String { // lower_snake_case
  void myMethod() {
    print('Hello');
  }
}
```

Compliant solution

```
extension MyExtension on String { // UpperCamelCase
  void myMethod() {
    print('Hello');
  }
}
```

Available In:
sonarcloud

